

Writeup: Favé

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup04_FavéOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `wqkG!0&QAb5D$omk`

Résumé

Write-up Favé

Informations générales

Fichier analysé : Favé

Objectif : Trouver le flag caché qui permet d'afficher "Good Job!"

Outils Utilisés

Outils utilisés

Radare2 pour l'analyse statique (r2, aaa, pdf, px, etc.)

CyberChef pour le décodage rapide (XOR)

Analyse Statique

Bonne nouvelle : le fichier n'est pas stripé, donc certaines informations sont encore accessibles.

1. Recherche de chaînes

Avec strings, j'ai repéré rapidement la présence des messages intéressants :

```
strings Favé | grep "Good"
```

Cela m'a permis de repérer :

```
"Good Job!\nBad Password!"
```

Analyse Dynamique

L'exécution du programme a permis d'observer son comportement et d'identifier les points de validation.

Identification du Mécanisme de Validation

Quelques commandes de base en bash

1. Première analyse

J'ai ouvert le fichier avec file pour vérifier son type :

```
file Favé
```

Il initialise une boucle de comparaison sur 16 caractères :

Chaque caractère de l'input est comparé avec un octet chiffré (encrypted_flag).

Chaque octet est déchiffré avec un XOR 0x55.

Découverte du Flag

Résultat :

ELF 64-bit, statically linked, not stripped

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.

Type

Type : ELF 64-bit LSB executable, statically linked, not stripped

Ouverture dans Radare2

Ouverture dans Radare2

J'ai ensuite ouvert le fichier avec Radare2 :

```
r2 ./Favé
```

Puis lancé une analyse basique

Puis lancé une analyse basique :

```
aaa
```

```
afl
```

J'ai vu que seule la fonction entry0 était trouvée, ce qui est normal pour des petits programmes.

1. Recherche autour des chaînes

Ensuite, j'ai recherché où la chaîne "Good Job!" était utilisée :

```
iz | grep "Good Job"
```

Elle se trouvait à l'adresse :

```
0x00402000
```

Puis j'ai cherché où cette adresse était utilisé :

```
axt 0x00402000
```

Résultat

Résultat : entry0 utilise cette chaîne.

1. Analyse du code avec pdf

Je suis allé sur entry0

Je suis allé sur entry0 :

```
s entry0
pdf
En étudiant le désassemblage, j'ai compris la logique suivante :
```

Le programme lit 32 octets en entrée.

Il vérifie que l'entrée a une longueur de 16 (0x10).

Schéma logique trouvé

Schéma logique trouvé :

```
for (i = 0; i < 16; i++) {
if (input[i] != (encrypted_flag[i] ^ 0x55))
erreur();
}
success();
```

1. Récupération de la donnée chiffrée

Avec Radare2, je suis allé à l'adresse du flag chiffré :

```
s 0x402018
px 16
```

Contenu lu

Contenu lu :

22 24 3e 12 74 65 73 04 14 37 60 11 71 3a 38 3e

7. Décodage avec CyberChef

j'ai utilisé CyberChef.

Entrée

Entrée : 22243e127465730414376011713a383e

Opérations

Opérations :

From Hex

XOR avec la clé 55

To Text

Résultat obtenu

From Hex

XOR avec la clé 55

To Text

Résultat obtenu :

wqkG!0&QAb5D\$omk

Résultat final

Résultat final

Quand j'ai lancé le binaire et donné ce flag, le programme m'a répondu :

Good Job!